

Use-case: Collect All Regular Rewards (Plot Point)
Primary actor: Player
Goal in context: To guide the main character (Pencil) to all of the “plot points” on the grid and gain score. Each plot point represents one point of a drawing, and as they are collected these plot points connect together to complete the final drawing, causing the player to win.
Preconditions: The game round has started; the main character (pencil) is alive (Graphite > 0); plot points are present on the board.
Trigger: The player identifies all plot points on the graph paper and collects each of them.

Scenario:

1. The player clicks the “Start Game” button and selects a stage, or clicks the “Continue” button to continue where they left off.
2. The player views the game board and identifies the location of a Plot Point
3. The player presses a directional key (UP, DOWN, LEFT, or RIGHT) to move the character
4. The system validates that the target cell is not blocked by a wall or barrier (e.g., ruler, protractor).
5. The system moves the pencil to the target cell.
6. The system decrements the graphite length by 1 to account for movement cost.
7. The system detects that the pencil occupies the same cell as a Plot Point.
8. The system removes the Plot Point from the board.
9. The system increases the player's score by the reward amount, and updates the score display on the UI.
10. The user collects all of the plot points.
11. The player navigates to the end cell, where a win animation is played.

Exceptions:

1. The selected stage is locked as they have not yet unlocked it. An error message appears letting them know they must complete the previous stage first.
1. Move is invalid or blocked: The target cell contains a barrier (wall, ruler, etc). The system prevents movement and the character remains in the current cell; Enemies still move.
2. Pencil sharpener collision: The target cell contains a moving enemy (pencil sharpener) who will hold the player for a set amount of ticks, dealing constant damage (graphite drainage) until the player escapes; see use-case: “Escape Pencil Sharpener”
3. Eraser collision: The target cell contains a moving enemy (Eraser); the system deducts a significant amount of graphite (health) immediately. If graphite drops below zero, see use case: “Game Over (Out of Graphite)”.
4. Graphite depleted: the movement cost in step 6 reduces the graphite length below zero; see use case: “Game Over (Out of Graphite)”
5. Punishment collision: The target cell contains a punishment (white-out puddle); the system deducts a large amount of graphite length; if graphite < 0, see use case: “Game Over (Out of Graphite)”

6. Time expired: The level timer reaches 0:00 during the attempt to move or collect the point; the System ignores the action and immediately triggers the loss condition. See use case: "Game Over (Time Out)."

Priority:	Critical, to be implemented after basic functions
When available:	Second increment
Frequency of use:	Frequent

Use-case: Game Over (Out of Graphite)
Primary actor: Player
Goal in context: To handle the depletion of the main character's life resource (graphite), stop the game session, and provide the player with options to retry or quit.
Preconditions: The game level is currently active; the pencil is on the board with graphite > 0.
Trigger: An event occurs (movement, eraser collision, white-out puddle, or sharpener trap) that reduces the graphite value to or below zero.

Scenario

1. The system calculates the new graphite value following a game event (e.g., the player moved a tile or collided with an eraser)
2. The system detects that graphite ≤ 0 .
3. The system immediately locks player input to prevent further movement
4. The system halts the game loop (stops the timer, pauses enemy pathfinding)
5. The system renders the death animation (e.g., the pencil sprite snaps in half or crumbles into dust)
6. The system displays the "Game Over" user interface overlay.
7. The system displays the final statistics (Time elapsed and score achieved)
8. The player clicks a menu button (e.g., "Restart Level" or "Main Menu").
9. The system executes the requested action (reloads the map assets or transitions to the title screen)

Exceptions

1. Simultaneous win: the event that reduced graphite to zero was the collection of the final plot point. The system checks the "win condition" flag first; if true, the system prioritizes the "level complete" sequence over the Game Over sequence.
2. Simultaneous healing: the event that reduces graphite to zero occurs in the same game tick that the player collects a lead refill or a graphite shard. The system prioritizes the "item collection" event, so the graphite is restored before the death check validates. The player survives with the restored amount minus the damage taken.

Priority: Critical, to be implemented after basic functions
When available: Second increment
Frequency of use: Frequent

Use Case: Escape Pencil Sharpener
Primary Actor: Player
Goal in context: To escape the Grab state of the pencil sharpener enemy to minimize graphite loss.
Preconditions: The player is navigating the board; a pencil sharpener enemy is present.
Trigger: The pencil collides with a pencil sharpener sprite.

Scenario

1. The system detects a collision between the pencil and the sharpener.
2. The system immobilizes the player, and movement inputs are disabled.
3. The system initiates a “grind” state, decrementing graphite at a constant rate per tick.
4. The player rapidly presses the “space” key (or random directional keys, depending on implementation)
5. The system registers each key press and increments an internal escape counter.
6. The escape counter reaches the required threshold.
7. The system releases the player, restoring movement control.
8. The system applies a brief stun/invulnerability cooldown to prevent immediate re-capture.

Exceptions

1. Graphite depleted during hold: the constant damage reduces graphite to 0 before the player escapes. The system triggers the “game over” state.

Priority: Low, to be implemented after core gameplay
When available Third increment
Frequency of use Frequent

Use Case: Suspend and Resume Game
Primary Actor: Player
Goal in context: To exit a game session mid-match without losing progress and resume exactly where the player left off using the "Continue" function.
Preconditions: A level is currently in progress; the main character is alive (graphite > 0); the level timer > 0:00.
Trigger: The player decides to stop playing temporarily and presses the pause or exit button.

Scenario

1. [Exit Sequence] The player presses the pause key (e.g., esc) during gameplay.
2. The system halts the game loop (freezes timer, player, and enemy movements).
3. The system displays the pause menu overlay.
4. The player selects the "save & quit" button.
5. The system captures the current state of all game entities:
 - Pencil: X/Y coordinates, current graphite and current size.
 - Progress: current score and list of all collected "plot points"
 - Enemies: X/Y coordinates of all erasers and sharpeners.
 - Environment: status of hazards (ink/white-out) and remaining items.
 - Global: time remaining on the clock
6. The system writes this data to a local save file (e.g., save.json)
7. The system transitions the screen to the main menu.
8. The system detects a valid save file exists and enables/highlights the continue button in the main menu (which was previously greyed out)
9. [Resume Sequence] The player clicks the continue button on the main menu.
10. The system reads the save file and validates the data integrity.
11. The system loads the specific level map and places all entities at their stored coordinates with their stored values (graphite, score, etc.).
12. The system darts the game loop immediately (or in a paused state, requiring a key press to unfreeze)

Exceptions

1. Save file corrupted: the system attempts to read the save file in step 10 but finds corrupted data. The system displays an error message, disables the continue button, and forces the player to start a new game.
2. Player fails level: if the player restarts the game after losing (graphite = 0), the save file is deleted. The continue button is disabled on the main menu; player must select new game.

Priority: Low, to be implemented after core gameplay
When available: Forth increment
Frequency of use: Semi-frequent

Use Case: Game Over (Time Out)
Primary Actor: Player
Goal in context: To enforce the level's time limit.
Preconditions: A level is currently in progress; the main character is alive (graphite > 0); the exit has not been reached.
Trigger: The level timer countdown reached 0:00.

Scenario

1. The system decrements the level timer for the current game tick.
2. The system detects that the level timer value is 0.
3. The system immediately disables all player input (movement and interaction).
4. The system freezes all moving entities in their current position.
5. The system displays the "time's up" overlay on the screen (and will play an audio cue depending on implementation).
6. The system transitions to the standard "game over" screen.
7. The system displays the partial score achieved up to that point.
8. The player selects "restart level" or "main menu".

Exceptions

1. Simultaneous win: the player connects the final plot point (or enters the exit) on the exact same game tick that the timer hits zero. The system prioritizes the win condition, and the level complete sequence is triggered.
2. Simultaneous death: the player takes lethal damage (graphite = 0) on the same tick the timer hits zero. The system triggers the standard "game over" screen.

Priority: Critical
When available: Second increment
Frequency of use: Frequent (secondary failure state)